

第一关挑战 - 作业感受

一、我是怎么完成这次挑战的

刚开始看到任务时，我对 SGLang、RadixCache、Mooncake trace 这些概念基本都不熟。所以我没有一上来就看论文，而是先把目标拆成几个能实际验证的小步骤：先把 SGLang 服务跑起来，再完成一次普通推理，之后再去处理 trace 和 workload。这样做的好处是，每完成一步都能知道当前环境到底有没有问题，而不是把所有东西一次装完以后再一起排错。

环境部分我使用了 WSL 下的 Ubuntu，并建立独立的 Python 虚拟环境。按照作业要求安装 SGLang 0.5.14 和 Ray 2.56.0 后，加载 Qwen/Qwen3-0.6B，先通过 `/v1/models` 检查模型是否正常，再用 `/v1/chat/completions` 发出第一次请求。这一步成功以后，我才开始做 Mooncake trace。整个过程里，我会把终端输出和报错交给 AI 帮我解释，但命令和结果都是自己实际执行和检查的；遇到版本、接口等问题时，也会再去对照官方文档，而不是直接照着 AI 的答案继续做。

workload 部分我先从 `mooncake_trace.jsonl` 中随机抽取了 10 条记录，主要使用 `input_length` 和 `output_length`。随后根据这些长度构造 synthetic prompt，并用 Poisson 到达过程生成请求到达时间。为了让前一个请求还没结束时后面的请求也能继续到达，我用 Ray task 并发发送 HTTP 请求。最后 10 条请求全部返回 200，实际输入输出 token 数也都和目标值一致，并记录了每条请求的 latency。做到这里以后，我才开始回头读 RadixAttention、PagedAttention 和 Mooncake 的资料。这时再看 Prefill、Decode、KV Cache、prefix reuse 这些概念，会比一开始直接读论文容易理解很多。

二、我觉得最困难的部分，以及我是怎么解决的

对我来说，最难的并不是把某一条命令敲出来，而是把“论文里的概念”变成一个真正能运行的实验。例如题目只说根据 `input_length` 和 `output_length` 构造 synthetic prompt，再用 Poisson 到达过程发送请求。如果只停留在文字层面，这句话看起来很简单，但真正实现时还要考虑怎么让输入的 token 数尽量准确、怎样保证输出长度、怎样模拟请求的随机到达、怎样让请求能够并发，以及最后记录什么指标。

一开始我对 Poisson 到达过程也有点模糊，后来才搞清楚：单位时间内到达的请求数服从 Poisson 分布时，相邻请求之间的时间间隔服从指数分布，所以代码里实际生成的是随机的 inter-arrival time。另外，Ray 的作用也不是单纯为了“用上题目要求的软件”，而是为了让这些请求真正像在线服务一样并行执行。如果每个请求都等前一个完全结束后再发送，就失去了模拟在线 workload 的意义。

还有一个比较容易混淆的地方是 PagedAttention 和 RadixAttention。我最开始会把它们都理解成“KV Cache 优化”，但继续读资料以后才发现两者关注的层次不同：PagedAttention 更像是在解决 KV Cache 的显存

怎么分配、怎么减少碎片；RadixAttention 更关注已经算过的 KV 能不能在不同请求之间继续复用。把这两个问题分开以后，我对整个推理流程的理解也清楚了很多。

三、这次从 0 到 1 的过程给我的启发

这次挑战让我感觉，所谓从 0 到 1 的科研或工程工作，并不是先把所有理论都学完以后再开始动手，而是需要在“查资料、做实验、看结果、再回头补理论”之间不断往返。很多概念只有在真的跑过一次以后才会有具体感觉。比如之前看到 Prefill 和 Decode 只是两个名词，但做完 workload 后，再看到一条请求有几千个 input token、几百个 output token，就能很自然地想到前者主要对应 Prefill，后者会影响 Decode 的循环次数。

我觉得类似的工作至少需要几种能力：第一是把一个模糊的大任务拆成可验证的小步骤；第二是能够阅读官方文档和论文，不完全依赖二手教程；第三是保留实验数据和过程，因为很多判断最后都应该由结果来支撑；第四是知道什么时候可以借助 AI 提高效率，什么时候必须自己验证。AI 在这次任务里对我最大的帮助是降低了陌生领域的入门成本，比如解释命令、梳理概念和辅助写脚本，但真正让我确认理解是否正确的，还是程序能不能跑通、接口返回什么、数据是否对得上，以及官方资料怎么写。

这次只是第一次接触大模型推理系统，我现在对 SGLang、vLLM 和 KV Cache 的理解还停留在系统层面，还没有深入。不过相比一开始只知道“部署一个模型”，现在至少能把一次在线请求从 API、调度、Prefill、KV Cache、Decode 一直到输出和指标记录串起来。我觉得这种先跑通、再理解、再继续深入的方式，对以后做项目或者科研会比较有帮助。